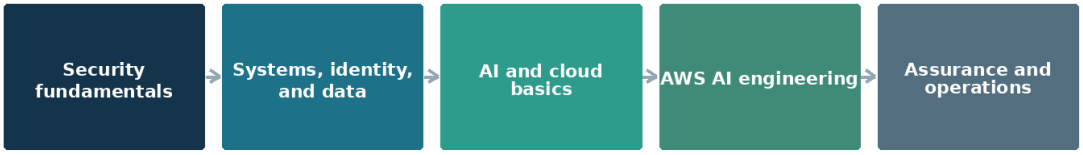
PRACTICAL SECURITY ENGINEERING MANUAL

Securing Enterprise AI on Amazon Web Services

A Practical Guide to Amazon Bedrock, SageMaker AI, IAM, Macie, Security Hub, OpenAI, Claude, RAG, MCP, Agents, Governance, and Incident Response

A practical learning path



Each stage adds capability without removing the controls learned earlier.

The manual begins with fundamentals and builds toward secure enterprise operation.

Alberto (Al) Leiva

Version 1.0 — July 2026

About This Manual

This manual explains how to design, build, secure, govern, test, and operate enterprise AI systems. It begins with security, systems, identity, data, AI, and cloud fundamentals. It then moves into Amazon Web Services and the services commonly used for enterprise AI. The goal is practical understanding: know what each component does, where trust changes, which controls matter, how to test them, and what evidence shows that they work.

INDEPENDENT EDUCATIONAL RESOURCE

This publication is not official Microsoft, OpenAI, Anthropic, OWASP, NIST, MITRE, ISO, or regulatory guidance. Product features, laws, standards, and licensing change. Verify important decisions against current official sources and qualified professional advice.

Safe and Authorized Practice

Use the exercises only in environments you own or are explicitly authorized to test. Confirm scope, accounts, targets, data, expected traffic, cost limits, communications, stop conditions, and cleanup before testing.

- Use synthetic or low-risk data whenever possible.
- Do not test public, third-party, customer, or production systems without written authorization.
- Do not attempt to obtain secrets, personal information, or access beyond the approved objective.
- Stop when an exercise affects an unintended system, person, or dataset.
- Protect findings and test evidence because they may contain sensitive details.

CORE RULE

Technical ability does not create permission.

How to Use the Manual

Read the foundation chapters in order. Later chapters can be used as a reference, but each assumes the reader understands identity, data flow, authorization, risk, and the AI system lifecycle. Complete the short laboratory at the end of each chapter and keep the evidence you create.

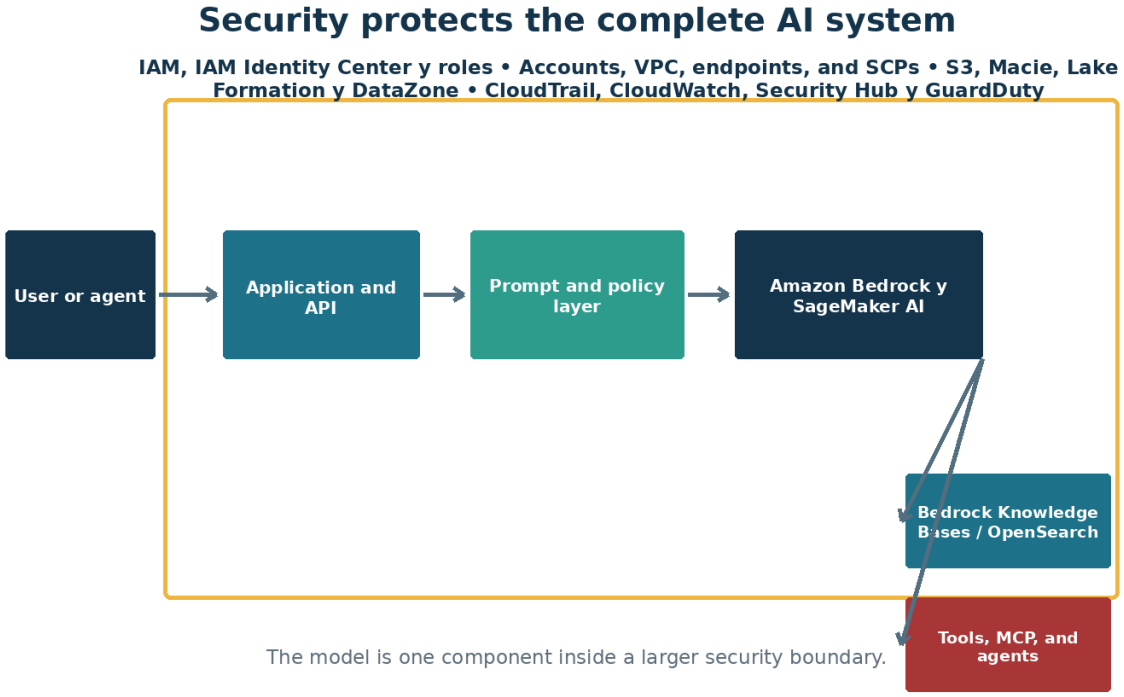


Figure 1. A model operates inside an application, identity, data, tool, and governance system.

Table of Contents

Chapter titles are clickable in Microsoft Word.

Learning Roadmap

The sequence moves from first principles to a complete enterprise implementation. Each part adds capability and responsibility.

Part I — Foundations — Understand the system before choosing controls.

Part II — Building AI on AWS — Build useful AI services with secure boundaries from the start.

Part III — Protecting AI Systems — Apply identity, data, application, model, and infrastructure defenses.

Part IV — Governance and Operations — Operate AI responsibly, measure risk, and preserve evidence.

Part V — Practical Laboratories — Turn concepts into repeatable engineering practice.

What the Reader Will Be Able to Do

- Explain an enterprise AI architecture and identify its trust boundaries.
- Use identity, network, data, application, model, and governance controls together.
- Build permission-aware RAG and bounded agent workflows.
- Apply OWASP LLM, agentic, NHI, API, and web guidance.
- Evaluate quality, safety, security, privacy, reliability, and cost.
- Collect evidence, respond to incidents, and map controls to major frameworks.

PART I — FOUNDATIONS

Understand the system before choosing controls.

UNDERSTAND • APPLY • TEST • PROVE

Chapter 1 — Security Engineering Fundamentals

Learn the language, habits, and core principles used to protect any system before applying them to AI or AWS.

What a security engineer protects

A security engineer helps the organization use technology while keeping risk within acceptable limits. The job is not to say no to every change. It is to understand what the organization is trying to accomplish, identify how it could fail or be abused, build practical safeguards, and make the remaining risk visible to the person authorized to accept it.

- Confidentiality: information is available only to authorized people and systems
- Integrity: information, software, and decisions are accurate and protected from unauthorized change
- Availability: services and information are usable when needed
- Privacy: personal information is handled lawfully and respectfully
- Safety: technology does not create unacceptable harm to people or the environment
- Accountability: important actions can be traced to a responsible identity

Risk in plain language

Risk is the possibility that a threat will exploit a weakness and cause harm. Start with the asset and business impact rather than a product feature. Describe the scenario clearly: who or what could act, what weakness could be used, what could happen, how likely it is, and which controls reduce the risk.

- Asset: something valuable that needs protection
- Threat: a person, event, condition, or capability that may cause harm
- Vulnerability: a weakness that can be used or triggered
- Control: a safeguard that changes likelihood or impact
- Residual risk: the risk that remains after controls
- Risk owner: the person accountable for the business outcome

Control types and evidence

Preventive controls try to stop an event. Detective controls reveal that it happened or is developing. Responsive controls contain and correct it. Recovery controls restore safe operation. A written policy is not proof that a technical control works. Evidence may include configuration exports, logs, approvals, tickets, test results, screenshots, code review, and incident exercises.

- Prefer controls that are repeatable and measurable

- Do not rely on training when a technical safeguard is practical
- Test that controls work under realistic conditions
- Record exceptions, owners, expiration dates, and compensating controls
- Preserve evidence in a protected and searchable location

A repeatable engineering method

Use the same basic method for every review: understand the purpose, inventory assets and dependencies, map data and trust boundaries, identify threats and failure modes, select controls, test them, document residual risk, monitor operation, and improve after change or incident.

- Ask simple questions until the architecture is clear
- Verify assumptions with configuration and logs
- Separate fact, inference, and opinion
- Escalate uncertainty when the potential impact is high
- Never test a system without permission and an agreed scope

A CONTROL THAT EXISTS ONLY ON PAPER

A policy requires administrator activity to be logged, but the cloud logging service is disabled in one subscription. The policy describes intent; the configuration and a successful test provide evidence.

Practice: Write a basic risk statement

- Choose an ordinary business asset such as a customer file repository.
- Identify one threat, one vulnerability, and one credible impact.
- Write the risk as a short cause-and-effect statement.
- Select one preventive, detective, responsive, and recovery control.
- Name the evidence that would show each control works.

Chapter Control Check

☐ Confidentiality, integrity, availability, privacy, safety, and accountability are considered.

☐ Risks are written as credible scenarios.

☐ Controls cover prevention, detection, response, and recovery.

☐ Control evidence is identified and retained.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and

remediation, approvals, exceptions, and the date and owner of the next review.

Chapter 2 — Systems, Networks, Identity, and Data Basics

Understand the technical building blocks that AI services inherit from ordinary applications and cloud systems.

Applications and APIs

An application normally has a user interface, business logic, data storage, and connections to other services. APIs let systems exchange requests and results. AI does not remove these layers; it adds a model and new forms of input, output, and uncertainty. Standard application weaknesses still matter.

- Validate input and encode output
- Authenticate the caller and authorize the requested action
- Use secure transport such as TLS
- Rate-limit requests and control resource consumption
- Handle errors without exposing secrets or internal details
- Keep dependencies patched and inventoried

Network concepts

Every connection has a source, destination, protocol, port, direction, and purpose. Public endpoints can be reached from the internet; private endpoints keep traffic on controlled networks. Firewalls, gateways, proxies, DNS, segmentation, and egress controls limit which systems can communicate.

- Document inbound and outbound flows
- Deny unnecessary paths
- Separate development, testing, and production
- Inspect and log important control points
- Protect administrative interfaces more strongly
- Plan DNS and certificate management early

Identity and access management

Authentication proves an identity. Authorization determines what that identity may do. Human users, applications, automation, service accounts, and AI agents all need identities. Roles and permissions should reflect job or task needs, and privileged access should be temporary and reviewed.

- Use multifactor authentication for people
- Prefer short-lived tokens over stored passwords or keys
- Apply least privilege and separation of duties

- Remove access promptly when purpose or ownership ends
- Review privileged and inactive identities
- Log successful and failed access decisions

Data lifecycle

Data is created or collected, classified, used, shared, stored, backed up, retained, archived, and deleted. Security must cover the entire lifecycle and every derived copy. AI creates additional derivatives such as chunks, embeddings, indexes, prompts, outputs, evaluation datasets, and logs.

- Collect only what is necessary
- Record source, owner, classification, purpose, and permitted use
- Encrypt sensitive data at rest and in transit
- Restrict access according to purpose
- Set retention and deletion rules
- Confirm that backups, indexes, and derived data follow deletion requirements

THE SECURE MODEL BEHIND AN INSECURE API

A model endpoint is private, but the public application API accepts any document identifier without checking whether the signed-in user may access that document. The model is protected; the business data is not.

Practice: Trace a simple web request

- Draw a user, browser, application API, identity provider, database, and logging service.
- Label every network connection and trust boundary.
- Describe authentication and authorization at each step.
- Mark where sensitive data exists or is copied.
- Add one control and one evidence source for each boundary.

Chapter Control Check

☐ Application and API controls remain part of AI security.

☐ Network paths and trust boundaries are documented.

☐ Authentication and authorization are treated separately.

☐ The data lifecycle includes derived copies and deletion.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

Chapter 3 — Artificial Intelligence, Machine Learning, and Language Models

Understand essential AI concepts well enough to identify security assumptions, limitations, and meaningful tests.

AI, machine learning, and generative AI

Artificial intelligence is a broad name for systems that perform tasks associated with reasoning, perception, prediction, or language. Machine learning finds patterns from data rather than relying only on hand-written rules. Generative AI produces new text, images, audio, video, or code based on patterns learned during training.

- A model is a learned mathematical system—not a database of guaranteed facts
- Training creates or adjusts a model from data
- Inference uses a trained model to produce a result
- Fine-tuning changes model behavior with additional examples
- RAG supplies selected external information at request time without retraining the model

How a language model handles text

A language model processes tokens, which are pieces of text rather than always complete words. The context window contains the instructions and information available for the current response. The model predicts useful continuations based on patterns and instructions; it does not verify truth unless the surrounding system gives it reliable sources and checks.

- Prompt: information or instructions supplied to a model
- System instruction: higher-priority guidance supplied by the application
- Token: a unit used to process input and output
- Context window: the limited working space for the request
- Temperature and related settings: controls that can influence output variation
- Hallucination: confident-looking content that is unsupported or incorrect

Embeddings and similarity

An embedding represents content as numbers so a system can compare meaning or similarity. Embeddings support semantic search and RAG. They are derived data and may still reveal information about the source. A vector database stores and searches these representations together with identifiers and metadata.

- Protect embeddings according to source sensitivity

- Use metadata to preserve ownership and permissions
- Evaluate whether similar results are actually relevant
- Remove embeddings when the source must be deleted
- Do not treat semantic similarity as proof or authorization

What models can and cannot do

Models can summarize, classify, transform, draft, extract, reason, and call tools when connected by an application. They can also misread context, follow malicious instructions, expose information, produce biased or unsafe content, and act with too much authority. The surrounding architecture must constrain these limitations.

- Use measurable acceptance criteria
- Verify sources for consequential answers
- Keep deterministic business rules outside the model
- Require human judgment for high-impact decisions
- Monitor behavior after deployment because use and attacks change

FLUENT IS MISTAKEN FOR CORRECT

An assistant produces a confident legal answer with a realistic-looking citation. The citation does not exist. The failure occurs because the workflow rewards fluent completion without retrieval, citation validation, or human review.

Practice: Observe model variability

- Ask an approved model the same harmless question several times.
- Record differences in facts, wording, uncertainty, and sources.
- Add trusted context and repeat the exercise.
- Define a test that distinguishes a supported answer from a fluent answer.
- List decisions that must never rely only on generated text.

Chapter Control Check

☐ Training, inference, fine-tuning, and RAG are distinguished.

☐ Tokens and context limits are understood.

☐ Embeddings are treated as protected derived data.

☐ Fluency is never treated as proof of correctness.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

Chapter 4 — Cloud and AI Service Fundamentals

Understand how shared cloud services, deployment choices, and the AI lifecycle shape security responsibility before entering AWS.

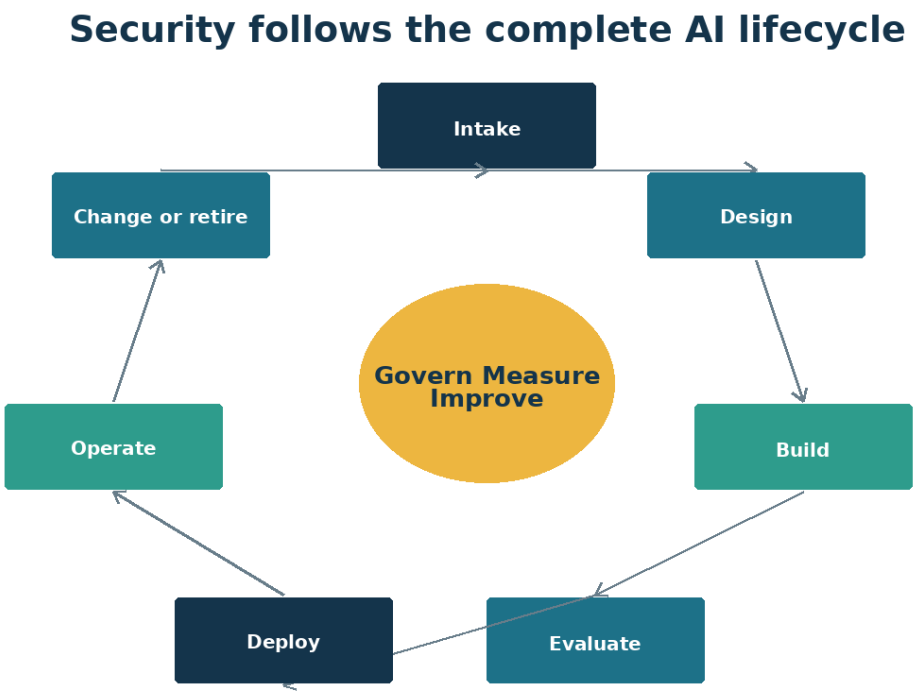


Figure 4. Security and governance follow the system from intake through retirement.

Cloud service models

Infrastructure services provide compute, storage, and networking with more customer control. Platform services manage more of the operating environment. Software services provide a complete application. Managed AI APIs and model platforms reduce infrastructure work but do not remove responsibility for identity, data, configuration, application logic, lawful use, or monitoring.

- Document what the provider secures and what the customer secures
- Review regional availability and data processing
- Understand administrative and support access
- Confirm logging, backup, recovery, and service-level commitments
- Plan how to leave or replace the service

Regions, tenants, and resource boundaries

A tenant represents an identity and administrative boundary. Subscriptions and projects organize resources, billing, policy, and access. Regions affect latency, resilience, legal obligations, and feature availability. Design these boundaries before data enters the platform.

- Keep production separate from experimentation
- Choose regions based on business and data requirements
- Avoid shared credentials across tenants or environments
- Tag resources with owner, purpose, classification, environment, and expiration
- Use policy to prevent unapproved configurations

The AI system lifecycle

An AI system moves through idea, intake, design, data preparation, development, evaluation, approval, deployment, monitoring, change, incident response, and retirement. Security and governance must follow the entire lifecycle. A control added only at deployment misses unsafe data and design choices made earlier.

- Approve the use case and prohibited uses
- Assess data and provider risk
- Threat-model architecture and misuse
- Evaluate quality, safety, security, privacy, and cost
- Monitor and retest after change
- Retire identities, data, indexes, endpoints, and contracts

From proof of concept to production

A proof of concept answers whether an idea can work. Production must also be secure, reliable, supportable, observable, affordable, compliant, and recoverable. Use a release gate with evidence rather than allowing a successful demonstration to become production by accident.

- Set expiration dates for experiments
- Use synthetic or low-risk data first
- Separate development and production access
- Define production acceptance criteria
- Assign operations and incident-response ownership

THE DEMONSTRATION THAT QUIETLY BECAME PRODUCTION

A pilot assistant gains real users and sensitive data while still using a developer's API key and test logging. No formal decision marks the change, so production controls and ownership never arrive.

Practice: Create an AI lifecycle gate

- List each lifecycle stage from idea through retirement.
- Assign an accountable owner to every stage.
- Define required security and governance evidence.
- Choose clear release and stop conditions.
- Add an expiration and cleanup process for experiments.

Chapter Control Check

☐ Cloud shared responsibility is documented.

☐ Tenant, region, environment, and resource boundaries are intentional.

☐ Security follows the complete AI lifecycle.

☐ Production requires explicit evidence and approval.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

Chapter 5 — How Enterprise AI Systems Work

Build a shared mental model of an AI system, its dependencies, and the boundaries a security engineer must protect.

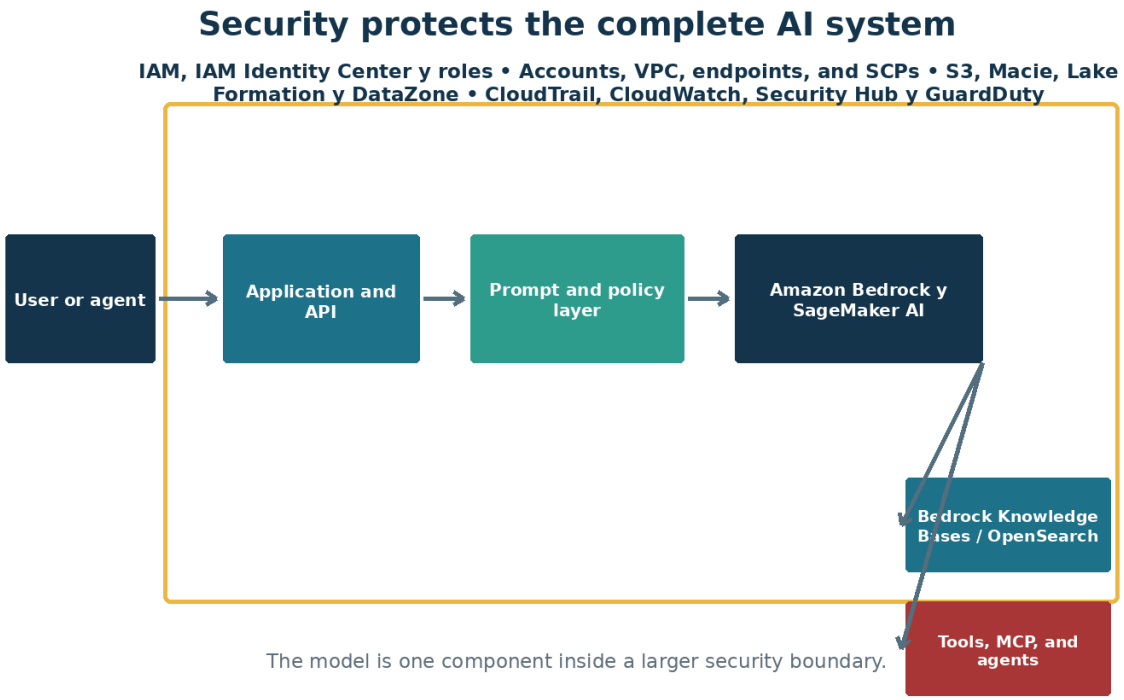


Figure 2. The model is only one part of the enterprise AI attack surface.

The system is larger than the model

A language model is only one component. A production service also contains an application, identities, prompts, data sources, retrieval services, tools, network paths, logs, administrators, and users. Security failures commonly occur where these components connect. Begin every review by drawing the complete system and naming the owner of each part.

- Model endpoint and deployment configuration
- Application and user interface
- System prompts, templates, and conversation state
- RAG stores, indexes, connectors, and source documents
- Agent tools, MCP servers, APIs, and downstream systems
- Identity provider, secrets, network controls, logging, and monitoring

Probabilistic behavior changes assurance

Traditional code follows defined logic. Generative AI can produce different answers to the same request, misunderstand instructions, or combine trusted and untrusted content. Testing therefore needs representative datasets, repeated runs, measurable thresholds, and ongoing monitoring. A demonstration that worked once is not evidence that the system is safe.

- Separate functional quality, safety, security, privacy, and reliability tests
- Record model name, model version, prompt version, parameters, and dataset version
- Test normal, unusual, hostile, and ambiguous inputs
- Define what the system must refuse, escalate, or hand to a human

Roles and shared responsibility

Cloud providers secure their platform, but the customer remains responsible for configuration, identity, data, application logic, legal use, and operational decisions. Model providers do not know the business context or acceptable harm. Assign accountable owners for the use case, data, model, security, privacy, operations, and human oversight.

- Business owner approves purpose and risk tolerance
- Data owner approves sources and access rules
- Engineering owner maintains the application and deployment
- Security validates architecture, controls, monitoring, and response
- Governance records decisions, evidence, exceptions, and reviews

A HELPFUL ASSISTANT BECOMES A SECURITY INCIDENT

A policy assistant answers correctly during a demonstration. In production, it retrieves a confidential investigation report because the vector index does not preserve document permissions. The model performed as designed; the authorization architecture failed.

Practice: Map a complete AI system

- Choose a low-risk internal assistant use case.
- Draw users, applications, models, data sources, indexes, agents, tools, identities, networks, and logs.
- Mark trust boundaries and every place data changes hands.
- Name the owner and required evidence for each component.
- List three ways the system could cause harm without being technically compromised.

Chapter Control Check

- ☐ The full system—not only the model—is documented.
- ☐ Trust boundaries and data flows are visible.
- ☐ Owners and approval points are named.
- ☐ Safety, privacy, security, and reliability are tested separately.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

Chapter 6 — AWS Foundations for AI Security

Create a governed AWS foundation so AI experiments do not become unmanaged production systems.

Organize the tenant and subscriptions

Use AWS Organizations, organizational units, accounts, naming standards, tags, service control policies, and AWS Config to make ownership and purpose visible. Separate production from development and testing. A single shared account makes cost allocation, access review, containment, and incident investigation harder.

- Use dedicated subscriptions or strong resource boundaries for production
- Tag owner, environment, data classification, cost center, and expiration date
- Apply service control policies, AWS Config rules, and Control Tower guardrails for allowed Regions, private networking, encryption, logging, and approved services
- Use resource locks only where they will not prevent emergency response

Identity before resources

Design access before deploying services. Use IAM Identity Center groups, permission sets, IAM roles, short-lived AWS STS credentials, and access reviews. Avoid broad AdministratorAccess policies and wildcard permissions. Prefer identity federation and short-lived tokens over stored credentials.

- Separate human administration from application access
- Use just-in-time privileged access
- Assign roles at the narrowest practical scope
- Create emergency accounts and test their procedures
- Log sign-ins, role changes, consent, and privileged operations

Network and platform baseline

For sensitive workloads, use private endpoints, controlled egress, network segmentation, DNS planning, API gateways, and centralized logging. Public access should be an explicit decision, not the default left behind after a proof of concept.

- Place Amazon API Gateway in front of externally consumed APIs
- Restrict storage, search, model, and secrets services with network controls
- Send diagnostic logs to a protected workspace
- Use AWS Security Hub CSPM findings, AWS Config results, and GuardDuty detections as input—not as the entire security program
- Define budgets, quotas, and alerts before testing at scale

THE FORGOTTEN PROOF OF CONCEPT

A team deploys a public model endpoint with a long-lived key and no budget alert. The demonstration ends, but the resource remains available. Months later the key is found in a repository and used for unauthorized inference.

Practice: Design a secure AWS landing zone

- Create a logical subscription and resource-group design for development, test, and production.
- Define required tags, service control policy outcomes, AWS Config rules, and Control Tower guardrails.
- Map administrator, developer, operator, auditor, and workload roles.
- Define allowed network paths and logging destinations.
- Write a cleanup and expiration rule for experiments.

Chapter Control Check

☐ Environments and duties are separated.

☐ Access uses groups, managed identities, and least privilege.

☐ Network exposure is intentional and documented.

☐ Logging, budgets, quotas, and cleanup rules exist before deployment.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

Chapter 7 — Zero Trust and Defense in Depth for AI

Apply familiar security principles to AI-specific data paths, tools, memory, and autonomous actions.

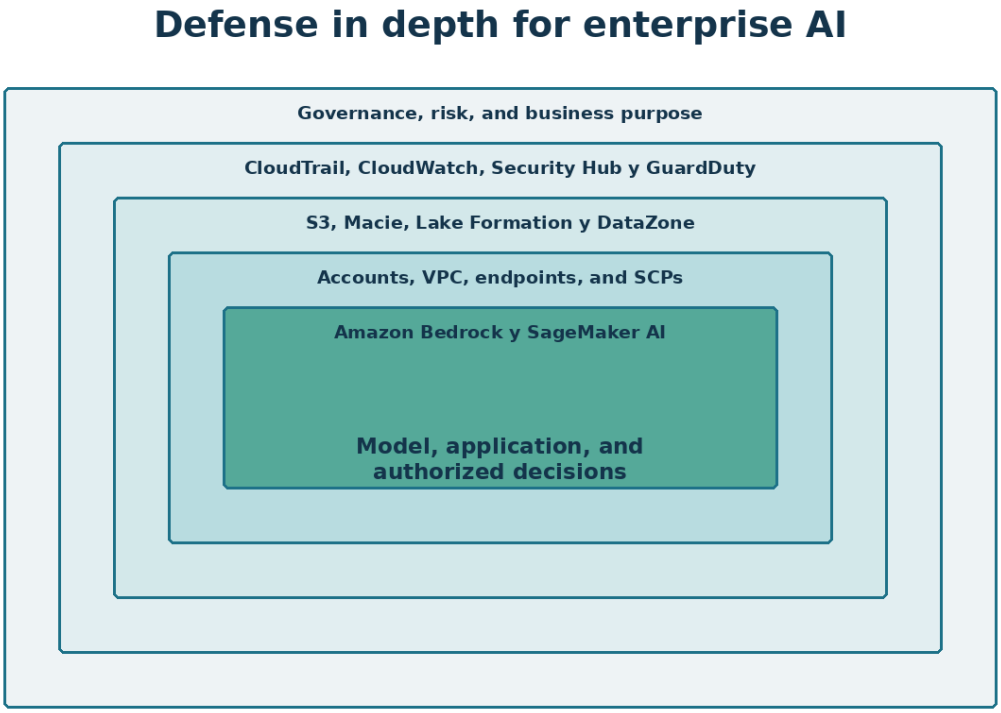


Figure 3. AI security depends on several independent layers.

Verify explicitly

Every user, workload, agent, tool, and data request needs a verified identity and an authorization decision. Do not treat a request as trusted because it came through a chat interface or because an agent generated it. The model is not an authorization engine.

- Authenticate at every service boundary
- Authorize the requested action and data—not only access to the application
- Recheck authorization when an agent calls a tool
- Preserve user context without granting the agent the user's full privilege
- Record actor, subject, resource, action, decision, and result

Use least privilege and assume compromise

Limit what a successful prompt injection, stolen token, compromised connector, or faulty agent can do. Narrow permissions, transaction limits, sandboxing, human approval, and environment separation reduce blast radius.

- Give agents task-specific tools instead of general shells
- Use read-only access unless modification is necessary
- Set spending, query, file, and transaction limits
- Require approval for deletion, payment, identity, legal, safety, and production changes
- Create a fast method to disable an agent, identity, connector, or model deployment

Layer the controls

No single filter can secure an AI system. Combine identity, network, data, application, model, monitoring, and governance controls. Content filters reduce certain harmful outputs but do not replace authorization, secure coding, or human oversight.

- Prevent with architecture and least privilege
- Detect with logs, behavioral signals, evaluations, and alerts
- Respond with practiced playbooks and revocation paths
- Recover with tested rollback, backups, and safe fallback services

A SAFE MODEL WITH AN UNSAFE TOOL

An agent has a well-tested system prompt but also has a tool that can delete cloud resources. A malicious document instructs the agent to call the tool. The correct defense is not a stronger sentence in the prompt; it is restricted permissions, input separation, approval, and transaction controls.

Practice: Build a defense-in-depth control map

- Select an agent that can search documents and open a service ticket.
- List preventive, detective, responsive, and recovery controls.
- State which controls remain effective if prompt defenses fail.
- Identify the maximum credible blast radius.
- Add one control that reduces that blast radius by at least half.

Chapter Control Check

☐ The model is never used as the authorization decision maker.

- ☐ Agent and tool permissions are task-specific.
- ☐ Consequential actions require stronger controls.
- ☐ Prevention, detection, response, and recovery are all represented.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

Chapter 8 — Threat Modeling AI and Agentic Systems

Find design weaknesses before deployment by modeling assets, data flows, trust boundaries, misuse, and failure.

Start with assets and harms

Protect more than confidentiality. Important AI assets include instructions, model access, embeddings, source integrity, user trust, decision quality, tool permissions, budget, safety, and the audit trail. Describe the harm that could occur if each asset is disclosed, changed, destroyed, or misused.

- Sensitive and regulated data
- System prompts, policies, and proprietary knowledge
- Model, dataset, embedding, and index integrity
- Agent credentials and delegated authority
- Correctness, availability, cost, reputation, and human safety

Map the real data and control flows

Show where prompts are assembled, where retrieved content enters context, where memory is stored, and where tools are invoked. Mark external content as untrusted even when it looks like ordinary text. Document administrative and maintenance paths as carefully as user paths.

- User-to-application and application-to-model flow
- Ingestion, chunking, embedding, indexing, retrieval, and deletion
- Agent-to-tool, MCP, API, and agent-to-agent calls
- Telemetry, evaluation, support, and incident-response paths
- Third-party model, connector, package, and SaaS dependencies

Model misuse and failure together

A secure review includes malicious attacks, ordinary mistakes, model limitations, unexpected combinations, and operational failures. Use STRIDE for conventional components, OWASP for application risks, MITRE ATLAS for adversarial AI techniques, and structured harm scenarios for people and business processes.

- Prompt injection, poisoning, extraction, evasion, and exfiltration
- Excessive agency and confused-deputy behavior
- Hallucination, bias, automation overreliance, and unsafe advice
- Outage, quota exhaustion, runaway cost, stale knowledge, and model retirement
- Abuse by insiders, administrators, suppliers, and compromised accounts

TRUST BOUNDARY HIDDEN INSIDE A PDF

A résumé-screening workflow retrieves text from uploaded PDFs. The architecture diagram shows the upload service but not the point where document text becomes model instructions. An attacker places hidden instructions in a PDF and changes the agent's behavior.

Practice: Threat-model a document assistant

- Identify assets, actors, entry points, dependencies, and trust boundaries.
- Write ten misuse or failure scenarios.
- Estimate likelihood and impact using the organization's method.
- Assign preventive and detective controls to the highest risks.
- Record residual risk, owner, due date, and acceptance authority.

Chapter Control Check

☐ Nontraditional assets such as trust, correctness, and cost are included.

☐ Retrieved content and tool calls appear as trust boundaries.

☐ Malicious and accidental failures are both modeled.

☐ Residual risk has a named acceptance authority.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

PART II — BUILDING AI ON AWS

Build useful AI services with secure boundaries from the start.

UNDERSTAND • APPLY • TEST • PROVE

Chapter 9 — Amazon Bedrock and SageMaker AI: Projects, Models, and Controls

Use Amazon Bedrock and SageMaker AI as a governed engineering environment rather than an isolated model playground.

Understand the platform role

Amazon Bedrock provides governed access to foundation models, Guardrails, Knowledge Bases, Agents, evaluation, and observability; SageMaker AI supports broader model building, customization, deployment, and MLOps. A project should have a defined business purpose, data classification, owner, environment, budget, and release path. Playground success is the start of engineering—not production approval.

- Create separate projects and connections for each environment
- Limit who can deploy models or change connections
- Use managed identities and approved connections
- Record region, provider, deployment type, quota, and content-filter configuration
- Export evidence of evaluations and approvals

Select models deliberately

Choose a model based on the use case and evidence. Larger or newer is not automatically safer or more appropriate. Compare quality, latency, cost, context limits, tool support, data handling, region, contractual terms, safety behavior, and portability.

- Create a representative evaluation dataset before selection
- Test at least one reasonable alternative
- Document model and provider limitations
- Confirm data processing and retention terms
- Plan for version changes, retirement, and fallback

Separate experimentation from release

Use a release gate that requires architecture review, threat model, data approval, identity review, evaluation results, red-team findings, monitoring, incident playbook, and business acceptance. Production changes should be traceable and reversible.

- Version prompts, code, model deployments, datasets, and evaluation configurations
- Require peer review for system instructions and tool definitions
- Protect production connections and secrets

- Retest after model, prompt, retrieval, tool, or policy changes

THE MODEL CHANGES BUT THE APPROVAL DOES NOT

A team replaces a model deployment to improve quality. The new version behaves differently with refusals and tool calls, but no regression tests run because the application code did not change.

Practice: Create a Bedrock or SageMaker AI release record

- Define a sample project and its owner, purpose, data classification, and environment.
- Compare two candidate models using five quality and five risk criteria.
- Create a release checklist and required evidence list.
- Define rollback triggers and the approved fallback model.
- Record the exact components that require retesting after change.

Chapter Control Check

☐ Every project has ownership, purpose, classification, and budget.

☐ Model selection is supported by evaluation evidence.

☐ Experiment and production duties are separated.

☐ Changes are versioned, retested, approved, and reversible.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

Chapter 10 — OpenAI, Claude, and Model Provider Choices

Understand deployment choices and avoid confusing a consumer application with an enterprise API service.

Distinguish the services

ChatGPT is an OpenAI application and workspace experience. The OpenAI API is a developer platform. OpenAI models may be consumed through the direct OpenAI API, while AWS model routes commonly use Amazon Bedrock or SageMaker AI under AWS controls. Claude may be obtained directly from Anthropic or through supported cloud platforms, including Amazon Bedrock where available. The security, billing, identity, retention, region, and feature model can differ by route.

- Document the contracting party and data processor
- Confirm whether customer data is retained or used for service improvement
- Verify region and feature availability
- Use enterprise workspaces and APIs for business data
- Do not assume a personal subscription has enterprise controls

Create a provider assessment

Assess providers and deployment routes using the same control questions. Review security attestations, data flow, subprocessors, incident notification, model lifecycle, abuse monitoring, administrative controls, logging, availability, portability, and exit support.

- Record allowed data classifications for each route
- Review authentication and key management options
- Confirm logs available to the customer
- Understand content filtering and appeal processes
- Test the effect of provider changes on application behavior

Design for controlled portability

Avoid tying critical authorization or business rules to one model's behavior. Use an application control layer, normalized evaluation datasets, provider-specific adapters, and documented fallback. Portability is not free, but controlled boundaries reduce emergency migration risk.

- Keep policy enforcement outside the model
- Maintain provider-independent test cases
- Separate prompts from source code where practical
- Track provider-specific features and limitations

- Test fallback for quality, safety, latency, and cost

BUSINESS DATA ENTERS A PERSONAL ACCOUNT

An employee uses a personal AI subscription to summarize a customer contract because the interface looks identical to the approved enterprise workspace. The missing control is not user intelligence; it is clear policy, technical discovery, training, and an easy approved alternative.

Practice: Compare deployment routes

- Compare direct OpenAI API, an Amazon Bedrock model route, direct Claude API, and Claude through Amazon Bedrock.
- Record identity, network, logging, retention, region, billing, and contract differences.
- Define allowed data classifications for each.
- Choose one route for a regulated internal assistant and explain the decision.

Chapter Control Check

☐ Chat applications and developer APIs are clearly distinguished.

☐ Provider and deployment-route risk is documented.

☐ Allowed data classifications are explicit.

☐ Fallback does not bypass security or governance controls.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

Chapter 11 — Prompt, Context, Memory, and Content Safety

Treat prompts and context as controlled application components while recognizing that they are not security boundaries.

Engineer system instructions

Write clear instructions that define role, allowed tasks, prohibited actions, escalation, source use, uncertainty, citations, and tool behavior. Store and version prompts like code. A system prompt can guide behavior, but users or retrieved content may still manipulate the model.

- Keep instructions concise and testable
- Separate untrusted content from instructions
- State when to refuse or ask for approval
- Require uncertainty and source disclosure where appropriate
- Never place secrets in system prompts

Control memory and context

Conversation history, long-term memory, retrieved passages, uploaded files, and tool results all become context. Define what may be stored, for how long, who may retrieve it, and how it is deleted. Do not let one user's memory influence another user's session.

- Classify memory content and set retention
- Prevent cross-user and cross-tenant recall
- Validate and label tool outputs
- Limit context to what is necessary
- Provide a method to correct or delete stored information

Use content safety as one layer

Content safety services can detect categories of harmful text or images and support configurable filters. They do not prove factual accuracy, prevent data leakage, enforce authorization, or stop all jailbreaks. Tune thresholds using the real use case and examine both false negatives and false positives.

- Test input and output filters separately
- Create safe fallback messages
- Log filter decisions without unnecessarily retaining sensitive content
- Establish review and appeal processes
- Retest after policy, model, language, or audience changes

THE SECRET IN THE SYSTEM PROMPT

A developer places a database credential in a system prompt and instructs the model never to reveal it. An attacker uses repeated extraction techniques and obtains the credential. Instructions cannot replace a secrets manager.

Practice: Create and test a system instruction

- Write a system instruction for a policy assistant.
- Define allowed tasks, refusal conditions, citations, uncertainty, and escalation.
- Create normal, ambiguous, hostile, and indirect-injection tests.
- Record pass criteria and unexpected behavior.
- Move every secret and authorization decision outside the prompt.

Chapter Control Check

☐ Prompts are versioned and peer reviewed.

☐ No secrets or final authorization decisions live in prompts.

☐ Memory has isolation, retention, correction, and deletion rules.

☐ Content filters are tested and treated as one layer.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

Chapter 12 — Secure Retrieval-Augmented Generation

Build RAG that improves answers without creating a new path around data authorization and retention controls.

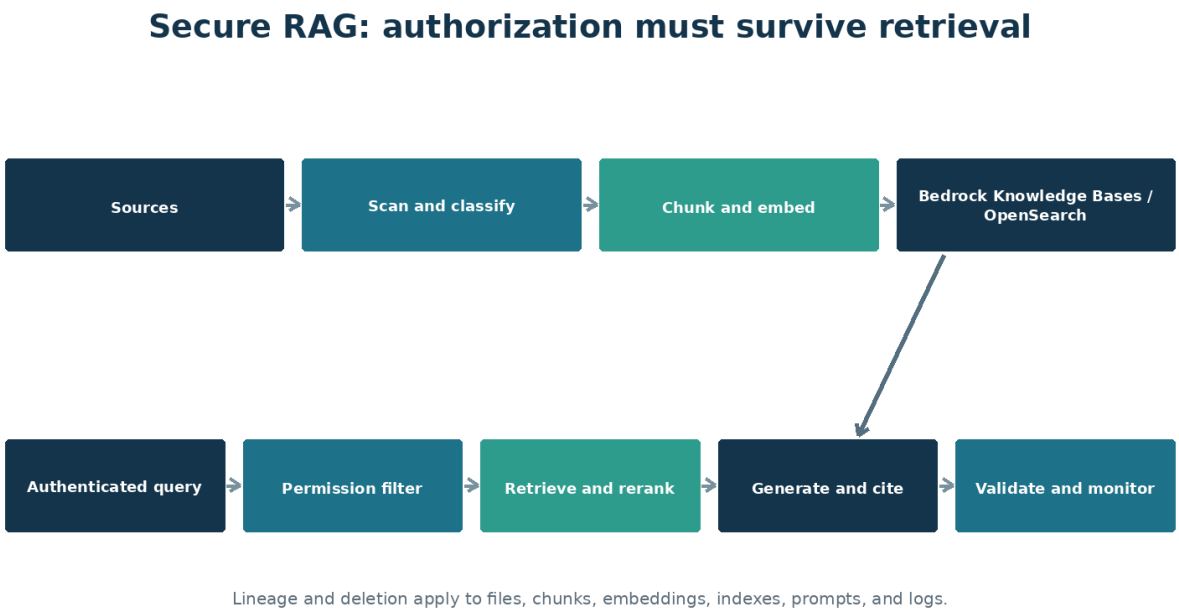


Figure 5. Secure RAG preserves authorization, lineage, and monitoring throughout the pipeline.

Understand the RAG pipeline

RAG retrieves relevant material and adds it to the model's context. It does not permanently teach the model or guarantee a correct answer. The pipeline includes ingestion, scanning, parsing, chunking, metadata, embeddings, indexing, querying, filtering, reranking, generation, citation, and deletion. Each stage needs an owner and controls.

- Maintain lineage from source document to every chunk and embedding
- Record source owner, classification, permissions, region, retention, and expiration
- Scan and validate files before parsing
- Keep source and index changes traceable
- Design deletion so derived content is removed

Preserve authorization during retrieval

Application access does not authorize every indexed document. Enforce document- or chunk-level access using the authenticated user, groups, tenant, purpose, and source permissions. Apply filtering before content reaches the model. Never ask the model to decide whether a retrieved passage is allowed.

- Use security trimming and deny by default
- Test cross-user and cross-tenant isolation
- Prevent metadata-filter bypass
- Use managed identities between retrieval components
- Log the query identity, filters, source identifiers, and authorization outcome

Defend the knowledge base

Treat every document as untrusted input. A document can contain hidden or visible instructions intended for the model. Separate retrieved data from system instructions, identify authoritative sources, restrict ingestion, detect unexpected changes, and test poisoned content.

- Approve connectors and ingestion sources
- Use checksums, provenance, and change review for critical content
- Detect indirect prompt injection and suspicious instructions
- Limit retrieved quantity and context
- Require citations that can be opened and verified

Evaluate retrieval and answers

Measure retrieval quality separately from generation quality. A fluent answer may be grounded in the wrong source. Use representative questions and evaluate precision, recall, context relevance, groundedness, completeness, citation correctness, refusal, latency, cost, and security behavior.

- Include questions with no answer in the corpus
- Test conflicting, stale, duplicated, and malicious documents
- Set minimum scores and manual-review rules
- Repeat evaluations after corpus, chunking, embedding, model, prompt, or ranking changes

DELETED DOCUMENT, SURVIVING EMBEDDING

A legal document is removed from storage under a retention request, but its chunks remain in the search index. The assistant continues to reveal portions because deletion was implemented only for the original file.

Practice: Threat-model and test a RAG assistant

- Create a small corpus with public, internal, and restricted documents.
- Define metadata and permission filters.
- Add one harmless document containing an indirect-injection test string.

- Test authorized retrieval, denied retrieval, unknown answers, citations, and deletion.
- Document every artifact required to prove access control and removal.

Chapter Control Check

☐ Source permissions survive ingestion and retrieval.

☐ Documents are treated as untrusted input.

☐ Lineage and deletion cover chunks and embeddings.

☐ Retrieval and generation are evaluated separately.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

Chapter 13 — Agents, Tools, MCP, and Agent-to-Agent Communication

Constrain autonomous and semi-autonomous systems so natural-language instructions cannot silently become unlimited authority.

Understand the agent loop

An agent observes, reasons, chooses a tool, acts, receives a result, and continues. Each loop can amplify an error or attack. Define a bounded goal, maximum steps, approved tools, time and spending limits, stop conditions, and actions that require human approval.

- Use narrow specialist agents instead of one all-powerful agent
- Set iteration, time, token, and transaction limits
- Validate tool arguments and results
- Prevent recursive delegation without limits
- Record a trace that connects instruction, decision, tool call, identity, and result

Secure tools and MCP

MCP standardizes how models discover and use tools and context. Treat every server, tool description, schema, returned resource, and authentication flow as a supply-chain and trust-boundary decision. An attractive tool description is not proof that the tool is safe.

- Approve and inventory MCP servers
- Authenticate clients and servers
- Pin versions and verify distribution sources
- Review tool descriptions for hidden or misleading instructions
- Allowlist tools and destinations
- Sandbox code execution and file access
- Require confirmation for consequential actions

Control delegation and agent-to-agent trust

An agent receiving a request from another agent must not assume the sender is authorized for the requested action. Authenticate both parties, preserve the originating user and purpose, constrain delegated scopes, and log the chain of authority.

- Use explicit trust relationships
- Avoid passing reusable broad tokens
- Set delegation depth and expiration

- Separate data exchange from executable instructions
- Detect circular work, task drift, and privilege accumulation

THE USEFUL BUT MALICIOUS TOOL

A developer installs an unreviewed MCP server that offers convenient document conversion. Its tool metadata instructs the model to send document contents to an external endpoint. The agent follows the description because the server was treated as trusted.

Practice: Review an agent design

- Define one business goal and the minimum tools needed.
- Assign a distinct identity and permission set to each agent and tool.
- List actions requiring confirmation.
- Set step, time, cost, destination, and transaction limits.
- Write shutdown and evidence-preservation procedures.

Chapter Control Check

☐ Agents have bounded goals, time, steps, cost, and permissions.

☐ MCP servers and tools are inventoried and approved.

☐ Delegation preserves identity, purpose, scope, and auditability.

☐ Consequential actions require deterministic controls or human approval.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

Chapter 14 — Amazon Q and Enterprise Productivity AI

Secure organizational use of Amazon Q and other productivity assistants where existing permissions and oversharing become AI risks.

Existing access becomes AI reach

A productivity assistant can rapidly discover and summarize information the user can already access. It may expose long-standing oversharing that was difficult to find manually. Before broad deployment, review SharePoint, Teams, OneDrive, mailbox, and group permissions and establish ownership for sensitive repositories.

- Discover overshared and unlabeled information
- Remove broad links and stale group memberships
- Apply sensitivity labels and DLP
- Prioritize executive, legal, HR, finance, security, and customer data
- Test with representative user roles

Govern agents and extensions

Amazon Q Business applications, plugins, connectors, actions, and indexed data sources expand capability and risk. Establish an intake process, approved environments, connector policy, owner and sponsor requirements, data classification, testing, publishing approval, monitoring, and retirement.

- Separate maker and production publisher roles
- Restrict connectors and data-loss paths
- Use environment and tenant policies
- Inventory published agents
- Review owners, permissions, usage, and inactivity

Manage adoption safely

Users need clear examples of allowed and prohibited information, how to verify answers, when human review is required, and how to report a problem. Blocking every tool without an approved alternative drives shadow AI.

- Provide role-specific training
- Publish an approved-tools directory
- Explain that generated content may be wrong
- Require source verification for consequential work

- Measure risky behavior and improve controls without treating education as surveillance

COPILOT FINDS WHAT SEARCH COULD NOT

A manager asks for salary-planning information. Copilot locates a spreadsheet shared through an old organization-wide link. Copilot did not bypass permissions; it made an existing access-control failure easier to exploit.

Practice: Prepare a productivity AI rollout

- Identify five high-risk repositories.
- Define permission, labeling, DLP, and ownership checks.
- Create agent publishing and connector approval rules.
- Write ten safe-use examples and five prohibited-use examples.
- Define adoption, risk, and incident metrics.

Chapter Control Check

☐ Oversharing is addressed before broad rollout.

☐ Agents, connectors, and extensions have lifecycle governance.

☐ Users can distinguish approved and personal services.

☐ Consequential outputs require verification and human responsibility.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

PART III — PROTECTING AI SYSTEMS

Apply identity, data, application, model, and infrastructure defenses.

UNDERSTAND • APPLY • TEST • PROVE

Chapter 15 — Identity Security and Non-Human Identities

Give every user, workload, application, and agent a governed identity with limited authority and a complete lifecycle.

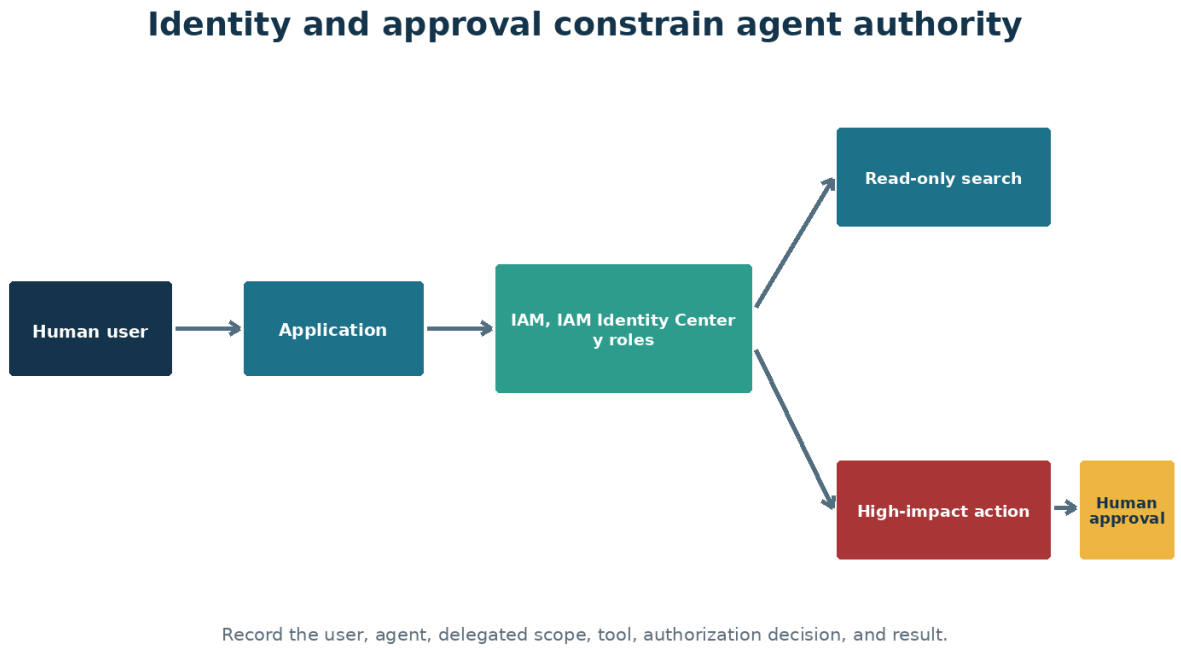


Figure 6. Agent identity and human approval limit delegated authority.

Choose the right identity

AWS IAM Identity Center manages workforce access, while IAM roles represent applications, services, automation, and agents. Workload roles and AWS STS remove the need to store many long-lived credentials. Choose an identity type that supports short-lived authentication, clear ownership, narrow authorization, and useful logs.

- Avoid shared user or service accounts
- Prefer managed identity or workload identity federation
- Use service principals only with documented ownership and purpose
- Use dedicated agent identities for governed agents
- Do not allow humans to perform routine work through non-human identities

Govern the NHI lifecycle

Non-human identities often outnumber people and are easy to forget. Every identity needs an inventory record, owner or sponsor, purpose, environment, credentials, permissions, dependencies, creation date, review date, and retirement trigger.

- Discover unknown and inactive identities
- Review permissions and recent use
- Rotate or eliminate long-lived secrets
- Separate development, test, and production identities
- Disable before deleting when dependencies are uncertain
- Remove identities, consent, roles, secrets, and trust relationships during offboarding

Apply OWASP NHI lessons

The OWASP Non-Human Identities Top 10 highlights improper offboarding, secret leakage, vulnerable third-party identities, insecure authentication, excessive privilege, insecure cloud configuration, long-lived secrets, weak environment isolation, identity reuse, and human use of NHIs. Treat this list as a design and audit checklist.

- Scan repositories, pipelines, images, and configuration for secrets
- Use certificates or federation instead of static keys where possible
- Assign the narrowest resource and API scopes
- Evaluate third-party agents and automation
- Alert on unusual sign-ins, locations, token use, and permission changes

Control delegated and autonomous access

An assistive agent may act on behalf of a signed-in user, while an autonomous agent may use application permissions. Record both the actor and the represented subject. Delegation must not silently expand the user's access, and autonomous permissions should be especially narrow.

- Distinguish delegated and application permissions
- Require consent and approval appropriate to impact
- Preserve user, agent, purpose, and tool identity in logs
- Use human approval for sensitive actions
- Create a tested emergency revocation path

THE OWNER LEAVES BUT THE AGENT REMAINS

An employee creates an agent with a broad application permission and leaves the company. The user account is disabled, but the agent identity and secret continue to operate because ownership and offboarding were never linked.

Practice: Create an NHI register

- Inventory five sample managed identities, service principals, pipelines, API keys, and agents.
- Record owner, purpose, permissions, credentials, environment, and last use.
- Identify reuse, overprivilege, long-lived secrets, and missing offboarding.
- Write remediation and emergency revocation steps.
- Define evidence for a quarterly access review.

Chapter Control Check

☐ Every NHI is unique, owned, purpose-bound, and inventoried.

☐ Short-lived or federated authentication is preferred.

☐ Environment isolation and least privilege are enforced.

☐ Offboarding removes identity, credentials, permissions, consent, and dependencies.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

Chapter 16 — Macie, DataZone, Data Governance, Privacy, and Shadow AI

Protect information across approved and unapproved AI use while preserving classification, purpose, retention, and evidence.

Know the data estate

Amazon Macie discovers sensitive data in Amazon S3, while AWS Lake Formation, AWS Glue Data Catalog, and Amazon DataZone support governed discovery, permissions, lineage, and data sharing. Begin with an inventory of sensitive sources, owners, permissions, locations, and AI uses. Classification without ownership or action becomes decoration.

- Use Data Map and catalog capabilities to understand sources and lineage
- Apply sensitivity labels that match business meaning
- Identify regulated, confidential, and high-impact data
- Record approved AI purposes and prohibited uses
- Review oversharing before enabling broad AI discovery

Apply data security controls

Information Protection, Data Loss Prevention, audit, eDiscovery, retention, Insider Risk, Communication Compliance, and DSPM for AI address different parts of the problem. Configure them according to risk, licensing, jurisdiction, and workforce expectations.

- Detect sensitive information in prompts and responses
- Restrict copying or uploading where risk requires it
- Monitor unusual access and sharing
- Preserve records and legal holds
- Protect investigation data and limit reviewer access
- Test policies in audit mode before enforcement when appropriate

Manage shadow AI

Shadow AI includes unapproved chat services, personal accounts, browser extensions, models, agents, connectors, and API keys. Combine discovery, policy, technical controls, education, and an easy approved alternative. A ban without usable tools encourages hidden work.

- Publish an approved-services directory
- Explain allowed data by service and account type

- Detect risky browser and network activity where lawful
- Offer a request path for new tools
- Measure adoption and risky behavior
- Respond proportionately and improve the system after findings

Privacy by design

Define purpose, lawful basis, notice, consent where required, minimization, access, correction, retention, deletion, cross-border transfer, and rights handling before processing personal information. Prompts, outputs, embeddings, logs, and human reviews can all contain personal data.

- Complete a privacy or data-protection impact assessment for higher-risk uses
- Avoid collecting data merely because it may be useful later
- Redact or tokenize where practical
- Limit production data in development and evaluations
- Test deletion across source, index, memory, logs, and backups

THE APPROVED TENANT AND THE PERSONAL BROWSER TAB

An organization licenses an enterprise AI service, but employees continue using personal accounts because the approved workflow is slower. Sensitive prompts therefore leave the governed environment.

Practice: Create an AI data protection plan

- Choose an AI use case and list every data source and derived data type.
- Assign classification, owner, purpose, access, region, retention, and deletion method.
- Define Macie discovery, Lake Formation permissions, DataZone governance, audit, and data-security-posture outcomes.
- Write employee guidance with safe alternatives.
- Test one access, one DLP, and one deletion scenario.

Chapter Control Check

☐ Data ownership, classification, lineage, and permitted purpose are recorded.

☐ Prompts, outputs, embeddings, memory, and logs are included.

☐ Shadow AI has discovery, policy, training, and approved alternatives.

☐ Privacy rights and deletion work across derived data.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

Chapter 17 — Infrastructure, Network, Secrets, and API Protection

Harden the ordinary cloud and application layers that carry AI traffic and authority.

Protect network paths

Use private endpoints for sensitive services where supported, control egress, segment environments, and route external access through managed gateways. Document DNS, certificates, firewalls, proxies, and dependencies. Private networking reduces exposure but does not replace identity or application authorization.

- Disable public network access when it is unnecessary
- Restrict outbound destinations for agents and tools
- Use web application and API protection
- Log accepted and denied flows
- Test from authorized and unauthorized networks

Manage secrets and keys

Use AWS Secrets Manager and AWS Key Management Service (AWS KMS) for secrets, keys, and certificates, but prefer designs that avoid secrets through IAM roles, workload federation, and short-lived AWS STS credentials. Separate duties for administration and use. Protect recovery and purge operations because deleting a key can destroy access to protected data.

- Never store keys in prompts, code, notebooks, images, or tickets
- Rotate credentials and test rotation
- Use narrow vault and object permissions
- Enable logging and appropriate recovery protection
- Inventory consumers before changing or retiring credentials

Secure APIs and tool endpoints

Use Amazon API Gateway or equivalent controls for authentication, authorization, schemas, quotas, rate limits, request size, allowed methods, versioning, and observability. Validate model-generated tool arguments exactly as untrusted user input.

- Use OAuth scopes and audience validation
- Reject unknown fields and unsafe destinations
- Apply idempotency for actions that may repeat
- Set per-user and per-agent quotas

- Return safe errors
- Do not place sensitive payloads in URLs or logs

Use Security Hub and posture management

AWS Security Hub CSPM findings, GuardDuty detections, Amazon Inspector vulnerability information, and workload protections provide useful visibility. Triage by exposure and business impact, verify applicability, assign owners, and track remediation. A dashboard is not evidence that risk is accepted or fixed.

- Enable relevant plans and diagnostic settings
- Review AI-related recommendations
- Integrate alerts with incident processes
- Validate remediation
- Document accepted exceptions and expiration

THE PRIVATE ENDPOINT WITH A PUBLIC SECRET

A model endpoint uses private networking, but its key is committed to a public repository. Network controls slow the attacker, yet the credential may still be used from an allowed path or through another compromised component.

Practice: Review a secure service path

- Draw user-to-API-to-model-to-search-to-storage connections.
- Mark public and private endpoints and outbound destinations.
- Choose identity and secret handling for every connection.
- Define API validation and limits.
- List logs and alerts needed to investigate one request end to end.

Chapter Control Check

☐ Public and outbound access are explicitly justified.

☐ Managed identity replaces stored secrets where practical.

☐ Model-generated API inputs are validated as untrusted.

☐ Posture findings become owned, verified remediation work.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

Chapter 18 — OWASP Risks for LLMs, Agents, NHIs, APIs, and Web Applications

Use the OWASP families together so AI-specific risks do not hide conventional application weaknesses.

OWASP LLM Top 10

The 2025 LLM list covers prompt injection, sensitive information disclosure, supply-chain risk, data and model poisoning, improper output handling, excessive agency, system prompt leakage, vector and embedding weaknesses, misinformation, and unbounded consumption. Map every item to architecture, tests, monitoring, and response.

- Prompt injection requires input separation, least privilege, approval, and monitoring
- Sensitive disclosure requires authorization, minimization, DLP, and output handling
- Supply-chain and poisoning risks require provenance, approval, integrity, and change detection
- Improper output handling requires encoding and validation
- Excessive agency requires narrow tools and deterministic limits
- Unbounded consumption requires quotas, rate limits, budgets, and anomaly detection

OWASP Agentic Top 10

Agentic systems introduce goal manipulation, tool misuse, identity and privilege abuse, unsafe memory, insecure communication, cascading failures, and human-agent trust problems. Review the current OWASP agentic guidance when designing or changing agents because this area evolves quickly.

- Bind agents to approved goals
- Authenticate agent-to-agent and agent-to-tool communication
- Control memory writes and retrieval
- Limit delegation depth and privilege
- Require human control over consequential outcomes
- Detect loops, drift, and coordinated failure

OWASP NHI, API, and Web guidance

AI applications remain web and API applications. Use the NHI Top 10 for machine identities, API Security Top 10 for exposed interfaces, Web Top 10 for application risks, ASVS for verification requirements, and SAMM for program maturity.

- Broken authorization remains a leading concern
- Injection includes SQL, commands, templates, and prompts

- Security misconfiguration affects cloud, application, and model services
- Dependency and integrity failures cross the entire AI supply chain
- Logging and alerting must connect application and AI events

Turn lists into engineering work

A Top 10 list is an awareness tool, not a complete threat model. Create test cases, control owners, evidence, and acceptance criteria for applicable risks. Record why a risk is not applicable instead of leaving it blank.

- Map risk to component and data flow
- Identify preventive and detective controls
- Create positive and negative tests
- Define severity and response
- Retest after material change

THE OUTPUT BECOMES CODE

A model summarizes a document and returns text containing script markup. The application inserts the output into a web page without encoding it. Prompt controls work, but improper output handling creates a conventional cross-site scripting risk.

Practice: Build an OWASP control matrix

- Select one RAG agent use case.
- Assess all LLM Top 10 items and relevant agentic, NHI, API, and web risks.
- Map each applicable risk to control, owner, test, evidence, and response.
- Run at least one safe negative test for five risks.
- Record residual risks and decisions.

Chapter Control Check

☐ LLM, agentic, NHI, API, and web risks are assessed together.

☐ OWASP lists complement rather than replace threat modeling.

☐ Every applicable risk has a control, test, owner, and evidence.

☐ Output is validated before another system interprets it.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

Chapter 19 — AI Supply Chain, Secure Development, and LLMOps

Protect models, data, code, prompts, packages, containers, pipelines, and deployment artifacts from creation through retirement.

Inventory the AI supply chain

An AI bill of materials should identify models, providers, datasets, embeddings, packages, container images, prompts, tools, MCP servers, APIs, evaluation sets, licenses, and deployment services. Record origin, version, integrity, owner, approval, environment, and known limitations.

- Use approved registries and repositories
- Verify checksums, signatures, and provenance where available
- Scan code, dependencies, images, and infrastructure templates
- Review model and dataset licenses
- Monitor provider advisories and end-of-life notices

Apply a secure development lifecycle

Use NIST SSDF principles and AI-specific practices throughout planning, development, build, release, and response. Protect source control, require peer review, separate duties, scan secrets, and keep production deployment controlled and repeatable.

- Threat-model before implementation
- Use branch protection and reviewed changes
- Generate repeatable infrastructure with Bicep or Terraform
- Sign and trace releases
- Require security, safety, quality, privacy, and cost gates
- Maintain a vulnerability and issue response process

Operate LLMOps with evidence

Version models, prompts, retrieval configuration, datasets, code, filters, policies, tools, and evaluation definitions. A release record should make it possible to reproduce what was tested and understand what changed.

- Use separate environments and approved promotion
- Maintain evaluation baselines
- Detect drift and regression
- Rollback quickly

- Retire endpoints, identities, indexes, data, and dependencies together

Control open-source and third-party models

Open-source models provide flexibility but increase customer responsibility for origin, integrity, license, hosting, patching, safety alignment, and monitoring. Third-party managed models shift some operations but still require due diligence and application controls.

- Obtain models from trusted sources
- Review model cards and limitations
- Test for backdoors, unsafe behavior, and unexpected capability
- Isolate evaluation environments
- Document accepted use and prohibited data

A CONVENIENT PACKAGE CHANGES THE PIPELINE

A developer installs a popular AI helper package. A compromised update reads environment variables during build and sends a deployment credential outside the organization.

Practice: Create an AI bill of materials

- Inventory a sample application's models, data, code, libraries, images, prompts, tools, APIs, and cloud services.
- Record provider, version, license, integrity, owner, and environment.
- Identify three dependencies with high replacement or compromise impact.
- Define monitoring and response for each.
- Create a release evidence checklist.

Chapter Control Check

☐ The inventory covers AI and conventional software components.

☐ Build and release paths use review, integrity, and separation of duties.

☐ Every production version is reproducible and reversible.

☐ Retirement removes connected data, identities, and services.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

Chapter 20 — Evaluation, Red Teaming, and Security Testing

Measure whether an AI system is useful, safe, secure, private, reliable, and affordable under realistic and hostile conditions.

Build an evaluation strategy

Start with the use case and harm scenarios. Create representative test data, expected behavior, scoring methods, thresholds, and review rules. Separate model quality from retrieval, safety, security, privacy, latency, reliability, and cost so a strong average does not hide a critical failure.

- Include common, rare, ambiguous, multilingual, and adversarial cases
- Test protected groups and accessibility needs
- Include cases with missing or conflicting information
- Define unacceptable failures regardless of average score
- Keep test data controlled and versioned

Red team safely

AI red teaming simulates misuse and adversarial behavior. It requires authorization, scope, safe targets, stop conditions, data handling, communications, and remediation ownership. Automated tools increase coverage but do not replace creative human testing or architecture review.

- Use isolated or nonproduction targets where practical
- Avoid real sensitive information
- Test direct and indirect prompt injection
- Test extraction, poisoning, tool abuse, excessive agency, evasion, and cost exhaustion
- Preserve prompts, responses, traces, configuration, and timestamps

Use tools responsibly

PyRIT, Amazon Bedrock model evaluation capabilities, garak, promptfoo, Giskard, and Inspect AI can support structured testing. Configure them only against systems you own or are authorized to test. Review tool documentation and generated traffic before execution.

- Record tool and version
- Use approved test accounts and limits
- Sanitize results before sharing
- Confirm findings manually
- Convert findings into regression tests

Make results actionable

A useful finding explains the scenario, preconditions, evidence, impact, affected components, likelihood, control gap, and recommended action. Assign an owner and due date, retest remediation, and document residual risk.

- Prioritize harm and exploitability over novelty
- Separate confirmed facts from possible impact
- Protect sensitive test results
- Track repeated failure patterns
- Report unresolved release blockers clearly

A PASSING AVERAGE HIDES A CRITICAL FAILURE

An assistant scores 96 percent on a quality dataset but reveals restricted information in two authorization tests. The average looks excellent; the system must not be released.

Practice: Design a safe AI evaluation

- Define five quality, five safety, five security, and five privacy tests.
- Set thresholds and release-blocking failures.
- Choose an approved tool or manual method.
- Run tests in a controlled target and preserve evidence.
- Write one finding and convert it into a regression test.

Chapter Control Check

☐ Evaluation covers quality, safety, security, privacy, reliability, and cost.

☐ Critical failures cannot be averaged away.

☐ Red teaming is authorized, scoped, limited, and documented.

☐ Confirmed findings become regression tests.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

PART IV — GOVERNANCE AND OPERATIONS

Operate AI responsibly, measure risk, and preserve evidence.

UNDERSTAND • APPLY • TEST • PROVE

Chapter 21 — Logging, Detection, Investigation, and Incident Response

Connect AI behavior to identity, data, application, network, and cloud evidence so incidents can be detected and contained.

Figure 7. Containment must preserve the context needed for investigation and recovery.

Design an end-to-end audit trail

A useful trace connects the human or workload identity, session, agent, model deployment, prompt template version, data sources, retrieved document identifiers, filters, tool calls, authorization decisions, content-safety events, response, and cost. Avoid logging sensitive content unless it is necessary and protected.

- Use correlation identifiers across services
- Synchronize time
- Protect log integrity and access
- Define retention by investigation and legal need
- Test that investigators can reconstruct one transaction

Detect meaningful behavior

Alert on unusual identity use, permission changes, secret access, query volume, denied retrieval, prompt-injection signals, policy bypass, new connectors, unexpected destinations, agent loops, tool failures, cost spikes, and changes to production configuration.

- Create baselines by user, agent, model, and application
- Tune alerts with operations and privacy stakeholders

- Prioritize combined signals
- Record investigation outcome
- Improve detection after incidents and red-team exercises

Prepare AI incident playbooks

Playbooks should cover sensitive disclosure, compromised NHI, prompt injection, poisoned RAG, malicious MCP or connector, harmful output, unsafe action, provider compromise, outage, and runaway cost. Define containment that does not destroy evidence.

- Disable or restrict agent and tool access
- Revoke tokens, secrets, consent, and sessions
- Quarantine sources, indexes, or connectors
- Roll back model, prompt, code, or policy
- Preserve traces and affected content
- Notify legal, privacy, safety, vendors, and customers as required

Recover and learn

Restore only after the cause and affected scope are understood. Validate controls and run regression tests before reopening. Conduct a blameless review that identifies technical, process, and organizational improvements.

- Confirm data and index integrity
- Test identity and permission changes
- Monitor closely after restoration
- Update threat models and playbooks
- Track lessons to completion

NO COMMON TRANSACTION IDENTIFIER

Security sees a suspicious sign-in, operations sees a cost spike, and the application team sees unusual tool calls. Without a shared correlation identifier, the events cannot be joined quickly.

Practice: Run a tabletop exercise

- Choose a poisoned-RAG or compromised-agent scenario.
- Identify first signals and decision makers.
- Practice containment, evidence preservation, communication, recovery, and notification.
- Record missing logs, permissions, contacts, and procedures.

- Assign improvements and schedule a retest.

Chapter Control Check

☐ One transaction can be traced across identity, application, model, data, and tools.

☐ Sensitive logging is minimized and protected.

☐ AI-specific playbooks exist and are exercised.

☐ Recovery requires validation and regression testing.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

Chapter 22 — Reliability, Resilience, Performance, and FinOps

Keep AI services dependable and affordable while preventing outages, loops, and resource abuse from becoming security incidents.

Design for failure

Models, providers, regions, networks, indexes, and dependent tools can fail or throttle. Define timeouts, retries with backoff, circuit breakers, queues, fallbacks, and manual procedures. Ensure a repeated request cannot duplicate a payment, deletion, or ticket action.

- Set recovery time and recovery point objectives
- Use idempotency for repeatable actions
- Test provider and region failure
- Define graceful degradation
- Maintain a safe mode that limits autonomous actions

Control consumption

Tokens, searches, tool calls, storage, evaluations, and monitoring generate cost. Denial-of-wallet attacks and accidental loops can exhaust budgets or service quotas. Establish limits at user, agent, application, project, and subscription levels.

- Set budgets and alerts
- Use rate, token, request, and concurrency limits
- Limit agent steps and tool calls
- Cache safely where appropriate
- Detect unusual spend and usage
- Assign cost ownership with tags and chargeback

Measure performance without weakening controls

Latency improvements must not bypass authorization, filtering, logging, or validation. Measure the whole user path and identify expensive or slow stages. Choose smaller models where they meet requirements and route tasks according to risk and complexity.

- Measure retrieval, model, tool, and network time separately
- Test load and quotas
- Protect caches against cross-user leakage
- Include security services in performance tests

- Reevaluate cost and quality after model changes

Use the Well-Architected perspective

Review AI workloads across reliability, security, cost optimization, operational excellence, and performance efficiency. Trade-offs should be explicit. A cheaper design that loses auditability or a secure design that cannot recover may still be unacceptable.

- Document nonfunctional requirements
- Run architecture assessments
- Assign improvements
- Retest after major change
- Record accepted trade-offs and owners

THE RETRY STORM

A tool API slows down. Hundreds of agents retry immediately, increasing load and cost until the dependency fails. Backoff, circuit breakers, queue control, and a bounded agent loop would contain the failure.

Practice: Create a resilience and cost plan

- Identify five dependencies and how each can fail.
- Define timeout, retry, fallback, and safe-mode behavior.
- Set user, agent, application, and subscription limits.
- Create cost and reliability alerts.
- Plan a controlled failure test.

Chapter Control Check

☐ Dependencies and failure modes are documented.

☐ Retries, fallbacks, and repeated actions are safe.

☐ Cost limits exist at several levels.

☐ Security controls remain active during performance and recovery events.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

Chapter 23 — Responsible AI, Human Oversight, and Organizational Change

Protect people and business decisions by designing accountability, transparency, accessibility, and meaningful human control.

Apply responsible AI principles

Fairness, reliability and safety, privacy and security, inclusiveness, transparency, and accountability should become testable requirements. A principle is useful when it changes design, data, evaluation, deployment, or oversight decisions.

- Identify affected people and possible harms
- Evaluate performance across relevant groups
- Provide understandable limitations
- Design accessible interaction and alternatives
- Assign accountable decision owners
- Document how concerns can be raised and corrected

Make human oversight meaningful

A human review step is not effective when the reviewer lacks time, information, authority, or a realistic way to disagree. Define which decisions require review, what evidence the reviewer sees, how uncertainty is shown, and how the reviewer can stop or reverse an action.

- Avoid rubber-stamp approval
- Show sources, confidence limits, and material assumptions
- Train reviewers on model limitations
- Record decisions and overrides
- Protect reviewers from production pressure that defeats the control

Manage automation bias

People often trust confident automated output. Require verification for legal, employment, health, safety, financial, identity, and security decisions. Measure whether users detect errors and provide clear escalation paths.

- Use calibrated language rather than false certainty
- Display citations that users can open
- Insert friction before consequential actions

- Sample and review outcomes
- Provide a non-AI path when necessary

Support adoption and reporting

Create simple guidance, role-specific training, approved examples, prohibited uses, and easy incident reporting. Encourage early reporting of mistakes without automatically treating every error as misconduct.

- Teach users how to protect sensitive data
- Explain how to verify generated content
- Publish approved services
- Give managers escalation and exception paths
- Use lessons from reports to improve design and training

HUMAN IN THE LOOP, ONLY IN NAME

A reviewer receives hundreds of AI-generated approvals each hour and sees no source evidence. The system records a human click, but the person cannot provide meaningful oversight.

Practice: Design an oversight control

- Choose a consequential AI-assisted decision.
- Define what the AI may recommend and what it may never decide.
- Specify evidence and uncertainty shown to the reviewer.
- Create override, appeal, and escalation paths.
- Test whether a reviewer can detect three seeded errors.

Chapter Control Check

☐ Responsible AI principles become measurable requirements.

☐ Human reviewers have time, evidence, authority, and alternatives.

☐ High-impact outputs are independently verified.

☐ Users can report, correct, appeal, and choose a non-AI path where needed.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

Chapter 24 — Legal, Regulatory, Intellectual Property, and Records Considerations

Recognize when engineering decisions create privacy, employment, consumer, copyright, contractual, sector, or records obligations.

Build legal review into intake

Requirements depend on jurisdiction, industry, data, people affected, and use. Security engineers should not give legal advice, but they should identify triggers early and provide accurate system information to legal and privacy teams.

- Personal and sensitive data
- Employment, credit, insurance, health, education, safety, and government uses
- Children and vulnerable groups
- Biometric, location, communication, and behavioral information
- Cross-border processing and data residency
- Automated or high-impact decisions

Address intellectual property

Review rights to training, retrieval, evaluation, and generated content. Record licenses for models, code, datasets, documents, images, and open-source components. Prevent users from assuming generated output is automatically original, accurate, or safe to publish.

- Respect source and model licenses
- Control ingestion of copyrighted and confidential material
- Provide citation and attribution where required
- Review generated code and content before use
- Establish rules for customer and employee content

Preserve records and evidence

Prompts, responses, approvals, model versions, evaluations, and agent actions may become business or regulatory records. Coordinate retention, legal holds, eDiscovery, privacy, investigation, and minimization. Keeping everything forever creates risk; deleting everything immediately destroys accountability.

- Classify records by purpose
- Set retention and access

- Protect legal holds
- Maintain chain of custody for investigations
- Document deletion and exceptions

Track changing requirements

Maintain a regulatory register and named owner. Map obligations to controls and evidence, review changes, and avoid claiming compliance based only on a vendor feature. Laws and standards may apply differently across regions and uses.

- Record applicability and interpretation owner
- Track implementation and evidence
- Review contracts and subprocessors
- Update assessments after material change
- Communicate limitations honestly

A TECHNICALLY SECURE BUT UNAUTHORIZED DATASET

A team protects a training dataset with encryption and access control but never confirms that the organization has the right to use the data for model development. Security controls do not create legal permission.

Practice: Create a requirements trigger list

- Choose a customer-support AI use case.
- Identify data, people, jurisdictions, decisions, contracts, and content rights involved.
- List questions for legal, privacy, records, and procurement teams.
- Map engineering evidence needed to answer them.
- Define events that require reassessment.

Chapter Control Check

☐ Legal and privacy review begins during intake.

☐ Data and content rights are verified.

☐ Records retention balances accountability and minimization.

☐ Compliance claims are supported by applicable controls and evidence.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and

remediation, approvals, exceptions, and the date and owner of the next review.

Chapter 25 — Frameworks: NIST, ISO, MITRE, and Control Mapping

Use complementary frameworks without turning implementation into a paperwork exercise.

NIST AI RMF

The NIST AI Risk Management Framework organizes work through Govern, Map, Measure, and Manage. Use it to establish accountability, understand context and harm, evaluate risk, and operate controls. The Generative AI Profile adds risks and actions specific to generative AI.

- Govern: policies, roles, culture, oversight, and accountability
- Map: context, users, impacts, dependencies, and risk
- Measure: tests, metrics, uncertainty, and monitoring
- Manage: prioritize, respond, communicate, and improve

ISO management systems and risk guidance

ISO/IEC 42001 provides an AI management-system structure. ISO/IEC 27001 addresses information security management, ISO/IEC 27701 privacy information management, and ISO/IEC 23894 AI risk guidance. Use official standards and qualified interpretation for formal conformity or certification work.

- Define scope and interested parties
- Establish policy, objectives, roles, competence, and documented processes
- Assess risks and impacts
- Operate controls and retain evidence
- Audit, correct, review, and continually improve

Cybersecurity and adversarial frameworks

NIST CSF 2.0 organizes cybersecurity outcomes across Govern, Identify, Protect, Detect, Respond, and Recover. NIST SP 800-53 supports detailed control selection. NIST SSDF and SP 800-218A support secure development. MITRE ATLAS catalogs adversarial AI tactics and techniques.

- Use CSF for program outcomes
- Use control catalogs for detailed requirements
- Use SSDF for the development lifecycle
- Use ATLAS and OWASP for threats and testing
- Avoid duplicate evidence by mapping one control to several requirements

Build a useful crosswalk

Start with implemented controls and evidence, then map them to framework outcomes. Do not create separate controls for every framework label when one well-designed control satisfies several needs. Record gaps and differences honestly.

- Control objective and description
- Owner and scope
- Implementation location
- Test method and frequency
- Evidence and retention
- Mapped requirements
- Exceptions and remediation

COMPLIANT IN A SPREADSHEET, WEAK IN PRODUCTION

A control matrix marks access reviews complete because a policy exists. The service principal permissions have never been examined. Framework mapping without operational evidence creates false confidence.

Practice: Map one control across frameworks

- Select quarterly review of agent and workload identities.
- Define objective, scope, owner, method, evidence, and failure handling.
- Map it to NIST AI RMF, NIST CSF, ISO/IEC 42001, ISO/IEC 27001, and OWASP NHI outcomes.
- Identify any framework-specific gap.
- Create a test record.

Chapter Control Check

☐ Frameworks are used for their intended purpose.

☐ Mappings point to real implementation and evidence.

☐ One strong control may support several frameworks.

☐ Formal certification relies on official standards and qualified assessment.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

Chapter 26 — Building an AI Governance and Assurance Program

Create an operating system for AI decisions, inventories, risk acceptance, monitoring, exceptions, and continual improvement.

Establish governance

Create a cross-functional group with business, technology, security, privacy, legal, compliance, risk, data, procurement, human resources, and safety representation as appropriate. Define decision rights so the group enables timely responsible use rather than becoming an unclear review queue.

- Approve policy and risk classification
- Set prohibited and restricted uses
- Define evidence and release gates
- Assign escalation and risk acceptance
- Review incidents, metrics, exceptions, and major changes

Maintain an AI system inventory

Inventory internally built, purchased, embedded, experimental, and shadow AI. Record purpose, owner, users, affected people, model, provider, data, agents, NHIs, integrations, region, risk tier, approvals, monitoring, incidents, review date, and retirement status.

- Discover through procurement, cloud, identity, network, endpoint, and data signals
- Reconcile inventory with technical sources
- Require registration before production
- Review inactive and high-risk systems
- Retire records only after connected assets are removed

Use risk-tiered assurance

Low-risk drafting should not require the same evidence as an autonomous system affecting employment or safety. Define tiers using impact, autonomy, data sensitivity, scale, external exposure, reversibility, and affected populations. Increase review and monitoring as risk grows.

- Document tier criteria
- Set minimum controls by tier
- Allow escalation when uncertainty is high
- Require independent review for high-impact uses
- Reassess after material change

Measure program health

Use a balanced set of indicators: inventory coverage, review time, overdue actions, high-risk exceptions, identity findings, data exposure, evaluation failures, incidents, time to contain, model changes, training, cost anomalies, and retirement completion.

- Avoid metrics that reward hiding incidents
- Report trends and material risk
- Connect findings to owners and deadlines
- Review whether controls reduce harm
- Improve policy and engineering standards after evidence

THE INVENTORY CONTAINS ONLY APPROVED SYSTEMS

Governance reports complete coverage because every registered application is reviewed. Network and expense data reveal employees using dozens of unregistered AI services. Inventory quality must be measured against discovery sources.

Practice: Design a governance workflow

- Create intake fields and risk-tier criteria.
- Define reviewers, decision rights, service targets, and escalation.
- List evidence required for low, medium, and high risk.
- Create change, exception, incident, and retirement workflows.
- Select ten program metrics that encourage honest reporting.

Chapter Control Check

☐ Decision rights and risk acceptance are clear.

☐ Inventory includes purchased, embedded, experimental, and shadow AI.

☐ Assurance effort scales with risk.

☐ Metrics measure coverage, outcomes, response, and improvement.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

PART V — PRACTICAL LABORATORIES

Turn concepts into repeatable engineering practice.

UNDERSTAND • APPLY • TEST • PROVE

Chapter 27 — Guided Security Engineering Laboratories

Practice the full workflow in safe, authorized environments using low-risk data and controlled targets.

Laboratory rules

Use only resources you own or have explicit permission to test. Begin with synthetic data. Set budgets and cleanup dates. Never direct automated testing at public or production systems without written authorization, approved scope, monitoring, and stop conditions.

- Document objective, owner, scope, accounts, resources, and expected traffic
- Create snapshots or reproducible templates
- Use separate test identities
- Protect test results
- Clean up endpoints, indexes, secrets, identities, logs, and data

Foundation labs

The first labs establish safe AWS multi-account structure, identity, data classification, and logging. Complete them before agent or red-team work.

- Create environment and resource naming design
- Build a least-privilege role matrix
- Configure an IAM role and Secrets Manager or AWS KMS access
- Design private service paths
- Send CloudTrail, CloudWatch, VPC Flow Logs, and service logs to central security accounts
- Create budget and quota alerts

AI application labs

Build a small policy assistant with approved documents, citations, permission-aware retrieval, content controls, and traceable requests. Add features one at a time and retest after each change.

- Deploy or select an approved model
- Create a versioned system instruction
- Build a protected RAG corpus
- Test unknown answers and citations
- Add a narrow read-only tool
- Create an agent identity and bounded loop

Assurance and operations labs

Test the completed application using threat modeling, OWASP cases, RAG poisoning, NHI review, red teaming, incident response, and framework mapping.

- Run evaluation and regression datasets
- Exercise direct and indirect prompt injection
- Review authorization and cross-user isolation
- Simulate compromised credentials and runaway cost
- Conduct an incident tabletop
- Collect an audit evidence package

A LAB WITHOUT CLEANUP

A learner completes a secure-agent exercise but leaves a model deployment, search service, public test endpoint, and application secret active. Cleanup is part of the lab, not an optional final note.

Practice: Prepare the lab charter

- State the learning objective and authorization.
- List resources, identities, data, regions, limits, and expected cost.
- Define stop conditions and evidence handling.
- Schedule cleanup and verification.
- Have a second person review high-impact tests.

Chapter Control Check

☐ Labs use owned or explicitly authorized targets.

☐ Synthetic or low-risk data is the default.

☐ Cost, traffic, and stop conditions are defined.

☐ Cleanup includes identities, data, indexes, endpoints, secrets, and logs.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

Chapter 28 — Capstone: Secure Enterprise Knowledge Agent

Bring architecture, identity, RAG, agent controls, testing, governance, and operations together in one evidence-based project.

Business objective

Design an internal knowledge agent that answers approved cybersecurity policy questions, cites sources, opens a service ticket through a narrow tool, and refuses unsupported or unauthorized requests. Use synthetic documents representing public, internal, and restricted classifications.

- Define intended users and prohibited uses
- Set measurable quality, security, privacy, safety, reliability, and cost outcomes
- Assign business, data, engineering, security, privacy, and operations owners
- Classify the use case and obtain approval

Architecture and build

Create separate development and production designs using Amazon Bedrock or SageMaker AI, an approved model, Bedrock Knowledge Bases or OpenSearch, Amazon S3, IAM roles, Secrets Manager and KMS, API Gateway, Macie and DataZone outcomes, Security Hub, GuardDuty, and centralized monitoring. Preserve document permissions through retrieval.

- Draw data and control flows
- Threat-model trust boundaries
- Use managed identities
- Restrict network and egress
- Version code, prompts, data, index, evaluation, and deployment
- Require human confirmation before ticket submission if the ticket changes service or access

Assurance

Evaluate normal questions, unknown questions, conflicting sources, stale content, direct and indirect prompt injection, sensitive disclosure, cross-user retrieval, tool misuse, excessive requests, harmful content, and provider failure. Convert confirmed findings into regression tests.

- Set release-blocking failures
- Run peer review and authorized red team
- Review NHIs and secrets
- Confirm logging and incident playbooks

1. Record residual risk and acceptance

Operations and evidence

Create dashboards, alerts, budgets, access reviews, evaluation schedules, change gates, incident playbooks, backup and fallback procedures, and retirement steps. Assemble a package that another engineer or auditor can follow without oral explanation.

- Architecture and threat model
- AI inventory and bill of materials
- Data and identity registers
- Evaluation and red-team reports
- Control matrix and approvals
- Operations, incident, recovery, and retirement procedures

THE CAPSTONE RELEASE DECISION

The agent meets quality and cost targets but fails one cross-user authorization test. The correct decision is to block release, correct the filtering design, rerun the complete relevant test set, and document the result.

Practice: Complete the capstone

- Create the project charter and architecture.
- Build the minimum working system.
- Implement identity, data, network, application, model, and monitoring controls.
- Run evaluation, red-team, resilience, and incident exercises.
- Present evidence, residual risk, and a release recommendation.

Chapter Control Check

☐ The capstone is useful, bounded, authorized, and measurable.

☐ Permissions are enforced before retrieval and tool action.

☐ Critical failures block release.

☐ Evidence supports build, test, operation, response, and retirement.

EVIDENCE TO KEEP

Architecture or data-flow updates, configuration or policy evidence, test results, findings and remediation, approvals, exceptions, and the date and owner of the next review.

APPENDICES

Reusable checklists, templates, glossary, and official learning sources.

UNDERSTAND • APPLY • TEST • PROVE

Appendix A — Production Readiness Checklist

Purpose and ownership

- ☐ Approved purpose and prohibited uses
- ☐ Business, data, engineering, security, privacy, and operations owners
- ☐ Risk tier and acceptance authority
- ☐ Human oversight and appeal path

Architecture and identity

- ☐ Current data-flow and trust-boundary diagram
- ☐ Least-privilege user, workload, and agent identities
- ☐ Managed identity or federation where practical
- ☐ Private and outbound network controls
- ☐ Secrets Manager, KMS, and rotation evidence

Data and RAG

- ☐ Approved sources, classifications, permissions, lineage, and retention
- ☐ Permission filtering before retrieval
- ☐ Poisoning and indirect-injection tests
- ☐ Derived-data deletion test
- ☐ Citation and unknown-answer behavior

Agents and tools

- ☐ Approved inventory of agents, MCP servers, connectors, and tools
- ☐ Bounded goals, steps, time, cost, destinations, and transactions
- ☐ Validation of tool input and output
- ☐ Human approval for consequential actions
- ☐ Emergency disable procedure

Assurance and operations

- ☐ Quality, safety, security, privacy, reliability, and cost evaluations
- ☐ Authorized red-team report and remediation

- ☐ End-to-end logs and alerts
- ☐ Incident, outage, cost, and provider playbooks
- ☐ Rollback, fallback, recovery, and retirement tests

Appendix B — AI Use-Case Intake Template

Use-case name and business objective

Response: _____

Executive sponsor and business owner

Response: _____

Intended users and people affected

Response: _____

Intended and prohibited uses

Response: _____

Decisions or actions supported

Response: _____

Models, providers, regions, and deployment route

Response: _____

Data sources, classifications, owners, and permitted purpose

Response: _____

RAG, memory, agents, MCP servers, tools, and external connections

Response: _____

Human oversight, appeal, correction, and non-AI alternative

Response: _____

Potential harms and highest credible impact

Response: _____

Legal, privacy, records, contractual, and sector review

Response: _____

Quality, safety, security, privacy, reliability, and cost measures

Response: _____

Risk tier, required reviewers, approval, and next review date

Response: _____

Appendix C — AI System and NHI Inventory Fields

AI system inventory

- System ID, name, description, and status
- Business owner, technical owner, data owner, and sponsor
- Users, affected people, scale, and geography
- Risk tier and approval record
- Models, providers, versions, deployment route, and region
- Data sources, classification, retention, and deletion
- RAG, memory, agents, tools, MCP, APIs, and connectors
- Security architecture, monitoring, incidents, exceptions, and next review
- Retirement date and confirmation of resource removal

Non-human identity inventory

- Identity ID, type, tenant, environment, and creation method
- Owner, sponsor, purpose, and dependent system
- Roles, API scopes, resources, and delegated authority
- Credential type, location, rotation, and expiration
- Last sign-in, recent activity, risk events, and review date
- Offboarding trigger, disable method, and emergency contact

Appendix D — Risk and Finding Template

- Title
- System and affected component
- Scenario and preconditions
- Threat or failure
- Weakness or control gap
- Business, technical, privacy, or safety impact
- Evidence and reproduction steps
- Likelihood and impact rating
- Applicable OWASP, MITRE, NIST, ISO, or internal requirements
- Recommended action and compensating controls
- Owner, due date, and status
- Retest result
- Residual risk and acceptance authority

WRITING TIP

State what was observed, what is inferred, and what could happen as separate ideas. Clear findings are easier to verify and act on.

Appendix E — Glossary

Term	Meaning
Agent	Software that uses a model to plan or select actions, often through tools or other services.
Agent identity	A governed identity representing an AI agent and its allowed access.
AI bill of materials	An inventory of models, data, code, prompts, tools, services, and dependencies used by an AI system.
AI system	The complete combination of model, application, data, people, processes, identities, tools, and infrastructure.
API	A defined interface that lets software services exchange requests and results.
Authentication	The process of proving an identity.
Authorization	The decision about what an authenticated identity may do.
Chunk	A portion of a document prepared for search, embedding, or retrieval.
Confidentiality	Protection against unauthorized disclosure.
Content filter	A control that detects or blocks configured categories of input or output.
Context window	The limited amount of instructions and information available to a model during a request.
Control	A safeguard that changes the likelihood or impact of risk.
Data lineage	A record of where data came from, how it changed, and where it moved.
Data poisoning	Manipulation of training, evaluation, retrieval, or other data to influence system behavior.
Embedding	A numeric representation used to compare meaning or similarity.
Fine-tuning	Additional model training used to adjust behavior for examples or a task.

Term	Meaning
Groundedness	The degree to which an answer is supported by the supplied source information.
Guardrail	A policy, filter, limit, validation, approval, or other constraint on system behavior.
Hallucination	Generated content that appears plausible but is unsupported or incorrect.
Human oversight	Meaningful human ability to review, stop, correct, or reverse an AI-supported action.
Inference	Using a trained model to produce a result.
Integrity	Protection against unauthorized or improper change.
Least privilege	Providing only the access needed for a defined task and time.
LLM	A large language model trained to process and generate language.
LLMOps	Practices for versioning, releasing, monitoring, and retiring language-model applications.
Managed identity	An Azure-managed identity that applications can use without storing ordinary credentials.
MCP	Model Context Protocol, a standard used to connect model applications with tools and context.
Metadata filtering	Using attributes such as user, tenant, classification, or source to restrict retrieval.
Model card	A description of a model's intended use, evaluation, limitations, and other important information.
Non-human identity	An identity used by software, automation, workloads, agents, or devices rather than a person.
Prompt	Text or other input supplied to guide a model's response.
Prompt injection	Input that attempts to alter model behavior or override intended instructions.

Term	Meaning
RAG	Retrieval-augmented generation, which supplies retrieved information to a model at request time.
Red teaming	Authorized testing that simulates misuse, attack, or failure to identify risk.
Residual risk	Risk remaining after controls are applied.
Risk owner	The person accountable for the affected business objective and risk decision.
Security trimming	Filtering search or retrieval results so an identity receives only authorized content.
Service principal	An identity used by an application or service in Microsoft Entra ID.
Shadow AI	AI services or capabilities used without required organizational approval or visibility.
System instruction	Application-provided guidance intended to shape model behavior.
Token	A unit of text processed by a language model.
Tool calling	A model application's structured request for external software to perform an operation.
Trust boundary	A point where data or control crosses between different trust or authority levels.
Vector database	A service that stores and searches embeddings and associated metadata.
Workload identity	An identity used by an application, service, container, or automation workload.
Zero Trust	A security approach based on explicit verification, least privilege, and assumed compromise.

Appendix F — Official Sources and Continuing Learning

Use current official sources when implementing controls or making compliance decisions. Product pages, standards, laws, features, and model availability change.

CLOSING PRINCIPLE

A strong security engineer stays curious, verifies assumptions, documents evidence, protects people, and improves the system after every change and lesson.

Amazon Bedrock documentation

Amazon SageMaker AI documentation

AWS Security Reference Architecture for AI

AWS Identity and Access Management documentation

IAM Identity Center documentation

AWS Security Hub CSPM and GuardDuty

AWS Well-Architected Generative AI Lens

Amazon Bedrock Guardrails

AWS Responsible AI Lens

OWASP Top 10 for LLM Applications

OWASP Top 10 for Agentic Applications

OWASP Non-Human Identities Top 10

OWASP API Security

MITRE ATLAS

NIST AI Risk Management Framework

NIST Generative AI Profile

NIST Secure Software Development Framework

NIST Cybersecurity Framework

OpenAI API documentation

OpenAI file search

Anthropic Claude documentation

ISO/IEC 42001 overview

Maintenance Record

Review this manual at least every six months and after major changes to Amazon Bedrock, SageMaker AI, IAM, Macie, or Security Hub, model providers, OWASP guidance, NIST publications, or applicable law. Record corrections and release notes in the publication repository.